

南開大學
Nankai University

南　开　大　学

计算机学院

编译系统原理实验报告

从 SysY 到可执行程序

LLVM IR、RISC-V 汇编与
MLIR 渐进降低实验

Lab1：基础要求与进阶探索

杨博　2410550

王禹博　2411210

指导教师：王刚

姓名、学号沿用所给模板，尚待本组确认；实际分工请在提交前补全。

2026 年 10 月 11 日

成员信息与分工

姓名、学号沿用所给模板，尚待本组确认；实际分工请在提交前补全。

成员	学号	实际分工记录
杨博	2410550	待填写：按实际完成工作记录填写
王禹博	2411210	待填写：按实际完成工作记录填写

本实验的代码、命令记录与报告位于仓库 `lab1/`。SysY 特性程序、手写 LLVM IR、手写 RISC-V 汇编和本地运行库均为本实验独立编写；未引入其他团队的词法、语法、优化或代码生成实现。AscendNPU 部分依据课程指定的官方文档开展探索，引用来源在文中列明。

摘要

为给后续自主实现 SysY 编译器建立可验证的语义与目标代码基线，本文首先以阶乘程序观察预处理、词法与语法分析、中间代码生成、优化、汇编和链接的完整过程，并通过宏修改、优化级别及调试选项考察输出变化。随后设计覆盖整数、控制流、递归、作用域、数组与浮点转换的 SysY 程序，独立编写等价 LLVM IR 和 RV64GC 汇编，链接本地 SysY ABI 运行库。在原生与 QEMU 7.0.0 环境中，100 组输入的九个构建变体共完成 900 次验证，均与独立数学期望一致。进阶部分分析官方 AscendNPU IR VecAdd 的 GM/UB 地址空间及搬运、计算语义，并实测社区 MLIR 的 Linalg、SCF、控制流、LLVM 方言到 LLVM IR 的降低链，完成 64 个元素校验。实验区分本机运行证据与 NPU 文档分析，说明运行库 ABI、短路副作用和浮点舍入是跨表示等价验证的关键。

关键词： SysY；LLVM IR；RISC-V；运行库 ABI；MLIR；渐进降低

目录

一、 引言与实验范围 **2**

二、 实验环境与复现方法 **2**

三、 基础实验一：编译完整过程 **3**
（一） 程序及各阶段输出 3
（二） 可控探索：SSA、优化、调试与宏 4

四、 基础实验二：多种表示的语义等价 **5**
（一） SysY 示例与覆盖范围 5
（二） 手写 LLVM IR：控制流、数组与数值类型 6
（三） 手写 RISC-V 汇编与运行库链接 7
（四） 验证结果与范围 7

五、 进阶实验：MLIR 与 AscendNPU IR **8**
（一） 官方 VecAdd 的抽象层级 8
（二） 本机可执行的逐层 lowering 观察 9
（三） 配置尝试、方言差异与限制 10

六、 结论 **10**

一、 引言与实验范围

编译器实现需要将源程序语义保持到目标指令。仅凭某一份生成代码能够运行，无法判断错误来自源程序、IR、目标代码还是链接环境。因此，本实验先建立可复现的观察链，再用独立编写的多种表示相互校验，为后续实验提供输入程序、语义期望与运行工具。

总体要求将大作业划分为预备工作、词法分析、语法分析、类型检查与 SSA 生成、优化以及目标代码生成。Lab1 对应第一项，要求研究现有编译工具、手写 IR 和汇编，并探索 MLIR；本文不将 Clang 作为后续自主编译器的实现，也不声称已经完成后续模块。以 Clang 将 `.sy` 文件按 C 读取只是本次实验的语义基线：通过命令行注入运行库声明，不修改 SysY 源文件的语言结构。

实验分为三组：一个阶乘程序用于编译阶段观察，一个综合 SysY 程序用于跨表示验证，一个 VecAdd 用于中端抽象层级观察。课程仓库未附独立的《SysY 语言定义》和《SysY 运行时库》文件，故示例按总体要求列出的特性及通用 SysY2022 接口设计；数组与浮点作为进阶语法示例，张量在 MLIR 部分讨论。

二、 实验环境与复现方法

项目	本次实测配置
宿主	Linux x86-64，Python 3.10
前端与 LLVM 工具	Clang 14.0.0、LLVM 14.0.0
目标	RISC-V RV64GC，LP64D，32 位 `int`、64 位指针
汇编器	已有 `riscv64-unknown-elf-as`，GNU Binutils 2.45
Linux 链接工具	补充 GCC 11.4.0 交叉驱动、glibc 2.35 sysroot
运行器	`/opt/qemu/bin/qemu-riscv64`，7.0.0
进阶工具	社区 MLIR 14.0.0；未发现 BiShengIR、CANN 或 davinci 设备节点
报告	XeLaTeX，基于仓库 ctexart 模板调整

表 1: 工具版本来自 `results/environment.json` 与 `advanced.json`

已有 `~/riscv-elf-toolchains` 是裸机工具链，包含 newlib，不能直接充当 Linux glibc sysroot。本实验复用其中的汇编器与 ELF 检查工具，补充 Linux 链接所需工具和库，生成静态 ELF 后在 QEMU 用户模式运行。仅能生成 ELF 并不表示操作系统 ABI 一致：裸机启动、newlib 系统调用与 Linux 用户程序的启动及系统调用机制不同。静态链接也不意味着裸机执行，它只是无需目标动态加载器。

PATH 中还存在 QEMU 4.1.1，因此脚本优先使用上述绝对路径，并检查版本包含 7.0.0。补充依赖解包到被 Git 忽略的 `.tools/`，未替换已有工具链。复现命令如下：

```
make -C lab1 all
# separately:
make -C lab1 test
make -C lab1 stages
make -C lab1 advanced
make -C lab1 report
```

脚本在每次构建中保存工具版本和命令；中间产物保存在 `results/stages/` 与 `results/mlir/`，验证逐项记录在 `verification.csv`。工具路径可通过 `QEMU`、`RV_GCC`、`RV_SYSROOT` 等环境变量覆盖。

三、 基础实验一：编译完整过程

(一) 程序及各阶段输出

阶乘程序读取 n，对 $0 \le n \le 12$ 输出 $n!$，超界返回状态 1，避免 32 位有符号整数溢出。$0! = 1$ 与循环初值相容。

Listing 1: 唯一的编译过程观察程序

```c
#include "sylib.h"
#define FIRST_FACTOR 2
#ifndef EXTRA
#define EXTRA 0
#endif
/* A single pipeline example; the macro variant adds one to the result. */
int factorial(int n) {
    int i = FIRST_FACTOR;
    int f = 1;
    while (i <= n) {
        f = f * i;
        i = i + 1;
    }
    return f + EXTRA;
}
int main(void) {
    int n = getint();
    if (n < 0 || n > 12) return 1;
    putint(factorial(n));
    putch(10);
    return 0;
}
```



图 1: 观察链。各步骤由 Clang 驱动与独立工具完成，运行库和启动文件在链接阶段加入。

阶段	关键命令选项	从结果观察到的对应关系
预处理	`-E`	展开头文件声明、去掉注释，`FIRST_FACTOR` 变为 2，`EXTRA` 变为 0；保留行标记
词法分析	`-Xclang -dump-tokens`	`while`、标识符、数值字面量与运算符成为 token，并带源位置
语法与语义	`-Xclang -ast-dump`	`WhileStmt`、`BinaryOperator`、变量绑定与类型、隐式转换进入 AST
IR 生成	`-S -emit-llvm -O0`	局部变量用 `alloca/load/store`；循环条件用比较和基本块跳转
优化	`opt -mem2reg`、`-O2`	栈变量提升到 SSA；进一步内联、简化、向量化等
后端与汇编	`-S --target=riscv64-linux-gnu`、`as`	指令选择、寄存器分配、栈帧构造之后，汇编器编码指令并生成符号/重定位
链接	`gcc -static`	合并本程序、运行库、libc 和启动对象，解析符号并应用重定位

表 2: 命令及实际证据，完整输出见 `results/stages/`

Clang 的驱动阶段定义可由 `-###` 检查 [1]。词法分析负责识别 token，语法分析构造结构，语义分析解析变量作用域、检查运算及调用类型；AST 中的 `DeclRefExpr` 关联声明，而 `ImplicitCastExpr` 表达从左值读取等隐式行为。LLVM IR 并非 AST 的文本替换：循环已经成为控制流图，源变量读写变成明确的存储访问。

LLVM 后端在 IR 之后执行目标相关的指令选择、寄存器分配与栈帧处理。`-S` 停在汇编文本，`-c` 停在未链接目标文件。对象中的 `getint/putint/putch` 仍是未定义符号，函数调用含 `R_RISCV_CALL_PLT` 等重定位；链接器把它们绑定到运行库实现。省略运行库的真实链接尝试报 undefined reference；加入运行库后可执行文件正常运行。因此，语法正确、IR 合法与链接成功是不同检查层。

(二) 可控探索：SSA、优化、调试与宏

为单独观察 mem2reg，使用 `-O0 -Xclang -disable-O0-optnone` 生成初始 IR，再执行 `opt -S -mem2reg -verify`。普通 `-O0` 的 `optnone` 属性会阻止常规优化 pass；关闭该属性只是让本次观察得以进行，并不等同于开启整个 O2 管线。阶乘的两个循环变量被提升为：

```
%.01 = phi i32 [ 2, %1 ], [ %6, %4 ]
%.0  = phi i32 [ 1, %1 ], [ %5, %4 ]
%3 = icmp sle i32 %.01, %0
br i1 %3, label %4, label %7
```

每个 phi 的输入与前驱边配对：第一次进入时取常量，回边进入时取更新值。SSA 的单次赋值约束针对 IR 名称，不意味着源语言变量不可多次赋值 [2]。

指标	O0	O2
x86-64 `factorial` 中 alloca 数	3	0
x86-64 `factorial` 中 load 数	6	0
x86-64 `factorial` 中 store 数	5	0
x86-64 `factorial` 中 phi 数	0	20
RV64 目标文件完整.text 大小/字节	194	98

表 3: 实测静态指标。IR 统计限定函数，.text 统计整个目标文件，不含运行库。

表3显示优化后显式栈访问消失，但 phi 数量反而增加。检查宿主 O2 IR 可见 `<4xi32>` 运算及 `llvm.vector.reduce.mul.v4i32`，说明循环被向量化并形成主循环与尾部处理。这不是“IR 条数越少越好”的例子。加入 `-fno-vectorize-fno-slp-vectorize` 关闭向量化后，输出回到标量形式；其文件为 `factorial_O2_novec.ll`。RV64GC 未启用 V 扩展，其 O2 汇编仍采用标量乘法。不同目标的优化结果不能直接相互套用，.text 变化也不能单独证明运行更快。

加入 `-g` 后，IR 增加 `DICompileUnit`、`DILocation` 等元数据，ELF 中出现 `.debug_info/.debug_line` 等节；真实行号映射见 `debug_lines.txt`。调试信息帮助将目标指令关联回源码，不属于程序的数值计算。调试与优化可同时启用，不过优化后的变量或语句可能合并、消失。

将 `EXTRA` 从 0 改为 1 后，预处理结果由 `returnf+0` 改为 `returnf+1`；同样输入 5 的两个版本分别输出 120 和 121。这一修改将宏展开与最终行为联系起来。两种优化级别分别验证 $n = 0, \ldots, 12$，共 26 次正确输出；对 $-1, 13$ 的 4 次测试均返回 1，宏变体另通过 1 次运行。

实际生成的 Clang 汇编最初含 `.addrsig/.addrsig_sym`，已有 GNU 汇编器无法识别。通过 Clang 选项 `-fno-addrsig` 关闭该地址显著性元数据后可正常汇编；无需手工改写生成文件。此问题说明文本指令语义一致并不保证不同工具支持完全相同的汇编伪指令。

四、 基础实验二：多种表示的语义等价

(一) SysY 示例与覆盖范围

`features.sy` 读取整数 n, d 及浮点 f，输出 11 行观测值。阶乘输入被夹到 $[0, 8]$；除余只在 $d \neq 0$ 时执行。设计目的在于让语言特性具有可观测结果，尤其让 `tick()` 更新全局计数，以检测短路行为。

特性	SysY 中的触发点	IR ／目标侧关键表达
声明、初始化、赋值、常量、注释	`LIMIT`、`hits`、局部 `sum`	全局初始化、存储；常量在手写 IR 中直接使用
全部整数算术及单目正负	`mix`、`n/d`、`n%d`、`+n+-d`	`add/sub/mul/sdiv/srem`；`addw/subw/mulw/divw/remw`
六种关系、逻辑非	`compare` 按条件累加位权	`icmp/select`；`slt/seqz/snez`
`if/else`、`while`、`break/continue`	`mix` 中跳过第二项、在第四项前退出	CFG 边、循环 phi；分支与回跳
函数、递归、void 返回	`fact`、`printline`	`call/ret`；参数寄存器与调用保存
嵌套作用域	块内 `constintn=7`	独立绑定；外层输入仍保持在后续运算中
一维与二维数组、数组形参	局部 `a[4]`、全局 `grid[2][3]`、`sumrow`	`alloca/getelementptr`；字节偏移与行步长 12
浮点声明、运算、转换、I/O	`transform`、`getfloat/putfloat`、`inttruncated=f`	`fmul/fadd/fsub/fdiv/sitofp/fptosi`；F 扩展指令与浮点 ABI

表 4: 特性覆盖表；源文件没有 include、指针声明或 for 等额外 C 语法

`compare` 用位权 $1, 2, 4, 8, 16, 32, 64$ 分别代表 $<, \leq, >, \geq, =, \neq, !n$，使每种比较的真假都可检查。关系表达式仅放在条件中，符合 SysY 中 Exp 与 Cond 的语法区别。浮点常量先绑定为 `constfloat`，以免 C 基线因无后缀字面量默认 double 而发生额外提升；每步按 binary32 运算，并以 `-ffp-contract=off` 禁止乘加收缩。

(二) 手写 LLVM IR：控制流、数组与数值类型

`features.ll` 由实验独立编写，不是将 Clang 输出复制为“手写 IR”。标量尽量直接用 SSA 表达；局部数组保留内存形式。LLVM 14 的 typed pointer 语法用于明确区分数组对象指针与标量元素指针，文件不固定 triple，由编译命令选择宿主或 RV64。

`mix` 的循环头包含索引与累加值的 phi。continue 边携带新的索引与原累加值；正常回边携带新的索引与更新后的累加值；break 直接走到出口。局部数组通过 `getelementptr[4xi32]` 找元素，二维数组形参使用 `[3xi32]*` 保留行步长。GEP 计算地址而不是读取值，读取需要单独 load。

短路表达式不能直接用普通位运算替代。手写 IR 把 `d==0` 的真边直接送到合流块；只有 `d!=0` 才求值 `n!=0`，且只有后者为真才调用 tick：

```
%dzero = icmp eq i32 %d, 0
br i1 %dzero, label %logicend, label %andleft
andleft:
  %nonzero = icmp ne i32 %n, 0
  br i1 %nonzero, label %andright, label %logicend
andright:
  %t = call i32 @tick()
  ...
logicend:
```

```
%logical = phi i32 [1, %entry], [0, %andleft],
                   [%bool, %andright]
```

因此 $(n, d) = (5, 0)$ 的逻辑结果是 1、计数是 0；$(0, 3)$ 的结果是 0、计数是 0；$(5, 2)$ 的结果是 1、计数是 1。结果和计数联合验证避免“布尔值正确但副作用错误”的漏检。

整数除法采用 `sdiv`，余数采用 `srem`；负数除法向零截断，余数与被除数同号。浮点函数逐步使用普通 `fmul/fadd/fsub/fdiv`，没有 `fast` 标志；偏置使用 `sitofp`，最终另输出 `fptosi` 转换结果。LLVM assembler 接受文件后仍执行 verifier，检查基本块终结、类型、支配关系及 phi 前驱约束。

(三) 手写 RISC-V 汇编与运行库链接

`features_rv64.s` 使用 GNU 汇编语法，独立实现同一程序。RV64 寄存器为 64 位，但 SysY int 为 32 位，故计算使用 `addw/mulw/divw/remw` 等字操作；数组元素以 `lw/sw` 访问，指针和保存的返回地址以 `ld/sd` 访问。

按照 LP64D 调用约定，整型参数/返回值使用 a 寄存器，浮点参数/返回值使用 fa 寄存器，栈在调用点保持 16 字节对齐 [3]。`transform(floatx,intbias)` 中 x 在 fa0、bias 在 a0，两类参数分别分配寄存器，不能简单按源码位置把 bias 放在 a1。main 把 n, d 保存在 s0、s1，把输入浮点保存在 fs0；这些寄存器在返回前恢复。递归 fact 保存 ra 和自身的 n，防止子调用破坏父调用状态。计算出的余数保存在 s2，跨过 putint/putch 调用后再输出。

浮点到整数的 `fcvt.w.s` 显式指定 `rtz`，对应向零截断；正常浮点运算指定 `rne`。默认的浮点舍入若误用于转整数，会把 -3.75 转成 -4，而 SysY/C 的期望是 -3。作用域遮蔽则直接把 7 传给 printline，之后继续用外层 n 的保存寄存器，保证绑定不会相互覆盖。

本地 `runtime/sylib.c` 是独立编写的 SysY ABI 运行库，提供整数、字符、浮点及数组 I/O 和计时入口。它不是其他团队的编译器源码，也不是从公开运行库复制而来。特性程序实际使用 getint、getfloat、putint、putch、putfloat；这些符号在目标对象中保留外部引用，链接时由运行库解析。putfloat 使用 `%a` 输出十六进制浮点，避免把十进制打印精度误当作计算误差。其计时实现只支持一个活动区间，本实验未调用计时接口，不以此实现替代课程未来可能给出的完整计时行为规范。

关键编译链如下，完整路径由脚本确定：

```
llvm-as features.ll -o features.bc
opt -verify -disable-output features.bc
clang --target=riscv64-linux-gnu -march=rv64gc \
  -mabi=lp64d -ffp-contract=off -c features.ll -o ir.o
riscv64-unknown-elf-as -march=rv64gc -mabi=lp64d \
  features_rv64.s -o asm.o
riscv64-linux-gnu-gcc -static -no-pie \
  asm.o runtime_rv64.o -o asm_rv64
/opt/qemu/bin/qemu-riscv64 asm_rv64
```

(四) 验证结果与范围

九个变体包括：SysY 基线与手写 IR 分别在宿主/RV64 的 O0/O2 构建，共 8 个；再加手写汇编的 RV64 构建。验证包含 12 组定向输入与 88 组固定种子随机输入。独立 Python 判据用阶乘数学函数和闭式 $\mathrm{mix}(n, d) = 2n + d$，不用复制示例中的循环；除法用绝对值整除加符号恢

复，避免 Python 向下取整与 C 向零截断的差别；浮点每步通过 32 位打包解包舍入。每次同时检查退出状态、stderr、11 行输出和跨变体字节一致性。

输入 (n, d, f)	阶乘	mix	逻辑值/计数	商/余数
$(5, 2, 3.5)$	120	12	1/1	2/1
$(5, 0, -2.75)$	120	10	1/0	0/0
$(0, 3, 0)$	1	3	0/0	0/0
$(-7, 3, -3.75)$	1	-11	1/1	-2/-1
$(7, -3, 4.25)$	5040	11	1/1	-2/1
$(9, 2, -100.25)$	40320	20	1/1	4/1

表 5: 部分定向输入结果。全量 11 行输出见 `verification.json`

实际结果为 100 组输入、900 次执行全部通过；数组行和恒为 15，局部遮蔽恒输出 7，之后的单目运算仍使用原输入。测试域包含零除数处理、正负除余、比较相等、夹取边界、带小数的正负浮点和很小的二进制浮点。随机域为 $n \in [-1000, 1000]$、$d \in [-20, 20]$、有限浮点约在 $[-100, 100]$。

这一结果是测试范围内的等价证据，不是全部 32 位输入的形式化证明。测试排除有符号溢出、INT_MIN/(−1)、NaN/无穷及越界的浮点转整数；这些情况若处理不当，不同工具对未定义行为的选择可能不同。示例中除零路径由源程序明确规定为输出 0/0，因此不执行有问题的除余指令。

五、 进阶实验：MLIR 与 AscendNPU IR

(一) 官方 VecAdd 的抽象层级

MLIR 允许方言在同一模块内共存，随着 lowering 逐步明确计算、控制流、存储与目标接口；所谓 LLVM IR“单层”不意味着 LLVM 没有多个优化 pass，而是其基础操作语义没有 MLIR 那样可扩展的多级方言体系 [4]。MLIR 中的 LLVM 方言仍是 MLIR 操作，必须翻译后才成为 LLVM IR 文本 [5]。

课程指定的官方示例对 16 个 i16 元素执行向量加法 [6]。输入输出在 GM 地址空间，三个临时 memref 位于 UB；两个 load 将输入搬入 UB，vadd 完成计算，store 再写回 GM。HACC 属性表明设备函数与入口。各缓冲区的逻辑数据量为 $16 \times 2 = 32$ 字节；物理对齐、复用与实际片上分配应由具体编译结果确定，不能仅从这个数推出最终 UB 占用。

层级	主要语义	本实验能观察到的证据
HFusion	融合与高层算子优化，保留命名算子信息	官方架构文档；VecAdd 快速示例已在 HIVM 层，未展示 HFusion 输出
HIVM 高层	Tile 级计算、搬运、地址空间等	VecAdd 参考输入中的 `hivm.hir.load/vadd/store` 及 GM/UB 类型
HIVM 硬件映射与降低	核内资源、存储计划、同步和指令映射	官方架构及 pass 说明；本机未执行，不构造虚假的 pass dump
LLVM 与设备二进制	对接低层编译与目标代码生成	官方快速示例命令；无本机 NPU 可执行结果

表 6: Ascend 专用路径的概念层级与证据来源

官方架构描述 HFusion 到 HIVM 的转换、HIVM 的资源映射以及向低层编译工具对接 [7]。这里需要区分层级关系和固定 pass 顺序：入口层、编译选项、目标型号不同，具体管线可以不同；本实验没有运行专用工具，不能给出声称实测的唯一顺序。特别是 GM 到 UB 的搬运不是普通 CPU 标量 load 的同义替换，它具有硬件存储层级含义；同步与缓冲区生存期也是代码正确性的组成部分。

(二) 本机可执行的逐层 lowering 观察

为了得到真实的逐层输出,另行独立编写同样 16 个 i16 元素相加的社区 MLIR 程序 `vecadd_linalg.mlir`。它使用普通 memref 和 Linalg generic 表达逐元素加法, 不带 GM/UB 语义, 也不冒充 Ascend 编译路径。输入已采用 buffer 形式, 因此这条实验链没有额外的 tensor bufferization 步骤。

文件	转换选项	观察到的变化
`00_linalg.mlir`	解析并打印原始模块	索引映射及 parallel 迭代器保留逐元素结构
`01_scf.mlir`	`-convert-linalg-to-loops`	generic 变为 `scf.for`，内部显式两次 load、一次 addi、一次 store
`02_cfg.mlir`	`-convert-scf-to-std`	循环变为块参数、`br/cond_br` 与回边，SSA 值由块参数传递
`03_llvm.mlir`	memref、arith、std 分别转 LLVM 并协调 cast	memref 被降低为描述符与指针运算，操作变为 `llvm.*`
`04_vecadd.ll`	`mlir-translate` 的 `--mlir-to-llvmir` 选项	输出真正 LLVM IR，可由 llvm-as、opt 与 Clang 处理

表 7: 全部文件由 MLIR14 实际生成，pass 前后输出保存在 `pass_trace.txt`

SCF 阶段的核心输出如下:

```
%c16 = arith.constant 16 : index
%c0 = arith.constant 0 : index
%c1 = arith.constant 1 : index
scf.for %arg3 = %c0 to %c16 step %c1 {
  %0 = memref.load %arg0[%arg3] : memref<16xi16>
  %1 = memref.load %arg1[%arg3] : memref<16xi16>
```

```
      %2 = arith.addi %0, %1 : i16
      memref.store %2, %arg2[%arg3] : memref<16xi16>
    }
    return
  }
```

在 CFG 阶段，循环归纳变量由入口常量及回边更新值传给同一个块参数，作用类似 LLVM phi；因此渐进降低保留的是控制流语义，而不是源码的文本形态。

`llvm.emit_c_interface` 生成 `_mlir_ciface_vecadd` 包装接口。秩为 1 的 memref 描述符包括 allocated 指针、aligned 指针、offset、一个 size 和一个 stride；不能把函数当成普通 `voidf(short*,short*,short*)` 调用。C 测试驱动按该描述符构造三个数组，链接翻译生成的 LLVM IR，在 O0/O2 下分别验证 16 个常规输入及 16 个 i16 回绕输入，共 64 个元素检查全部通过。回绕用无符号位模式作判据，与普通 C 有符号 int 溢出问题区分。

(三) 配置尝试、方言差异与限制

本机检查未找到 `bishengir-compile`、CANN 常见安装目录或 `/dev/davinci*`。官方说明端到端运行需要 CANN，源码构建还依赖 LLVM 及配套补丁 [8]。因此采用题目允许的文档探索，同时保留 CPU lowering 的可执行证据。

直接把官方风格输入交给社区 MLIR14，会先得到 `customop'func.func'isunknown`；这是版本语法差异，MLIR14 仍使用 `func` 及 Standard 控制流表示。仅把函数语法适配为 `func` 后，下一次真实解析失败发生在 `#hivm.address_space<gm>`，提示 unregistered dialect。两次诊断分别保存，避免把新语法不兼容误判为 NPU 方言缺失。允许未注册方言只能保留未知操作/属性，并不会获得它们的语义校验或 lowering 实现。

对装有 BiShengIR/CANN 的环境，参考输入可使用官方命令形式：

```
bishengir-compile vecadd_hivm.mlir \
  -enable-hivm-compile -o kernel.o
```

该命令在本机没有执行。具体 IR 打印选项应先由安装版本的 help 确认，再保存每个 pass 的输出；不能直接假定社区 MLIR14 的选项名与 BiShengIR 一致。设备二进制还需通过 CANN runtime 完成注册、内存分配、拷贝与 kernel launch，单有 kernel.o 并不等于完成正确性验证。仓库 `advanced/README.md` 保存了后续观测步骤与官方参考链接。

六、 结论

本实验以真实命令和输出串联了预处理、前端、中端、后端、汇编和链接，并通过 mem2reg、优化级别、调试信息和宏修改观察到程序结构与最终行为的关系。综合示例的 SysY、手写 IR 和手写 RV64 汇编在九个构建变体中完成 900 次验证，建立了覆盖核心语法、数组与浮点的测试基线。

进阶实验实际保存了社区 MLIR VecAdd 的逐层降低输出和 64 个元素校验；Ascend 部分完成官方 HIVM 示例、地址空间及层级差异分析，并如实记录专用环境缺失。后续自主编译器可复用输入程序、期望结果、运行库接口和 QEMU 流程，逐步将 Clang 基线替换为自研词法、语法、类型检查、SSA、优化和代码生成模块。短路副作用、phi 前驱、数组步长及混合参数 ABI 应成为这些模块的重点验收项。

参考文献

[1] LLVM Project. Clang Command Guide[EB/OL]. https://clang.llvm.org/docs/CommandGuide/clang.html. 访问日期：2026-10-11.

[2] LLVM Project. LLVM Language Reference Manual[EB/OL]. https://llvm.org/docs/LangRef.html. 访问日期：2026-10-11.

[3] RISC-V International. RISC-V ABIs Specification[EB/OL]. https://riscv-non-isa.github.io/riscv-elf-psabi-doc/. 访问日期：2026-10-11.

[4] Lattner C, Amini M, Bondhugula U, et al. MLIR: A Compiler Infrastructure for the End of Moore's Law[EB/OL]. 2020. https://arxiv.org/abs/2002.11054.

[5] LLVM Project. Targeting LLVM IR, LLVM 14 source documentation[EB/OL]. https://github.com/llvm/llvm-project/blob/llvmorg-14.0.0/mlir/docs/TargetLLVMIR.md. 访问日期：2026-10-11.

[6] 华为. AscendNPU IR：编译与执行示例 [EB/OL]. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/quick_start/examples_zh.html. 访问日期：2026-10-11.

[7] 华为. AscendNPU IR：架构设计 [EB/OL]. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/architecture_zh.html. 访问日期：2026-10-11.

[8] 华为. AscendNPU IR：构建安装 [EB/OL]. https://ascendnpu-ir.gitcode.com/zh_cn/sources/introduction/quick_start/installing_guide_zh.html. 访问日期：2026-10-11.